

JOBSHEET III

Mata Kuliah: Praktikum Pemrograman Berbasis Objek

Enkapsulasi dan Konstruktor

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom.



Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 2B

D-IV Teknik Informatika

Tanggal: 11 September 2026

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

65141

TAHUN 2026-2027

C. Langkah Kerja

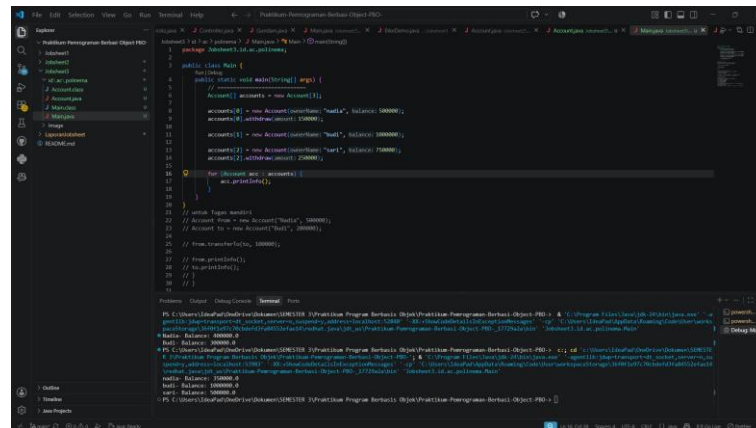
Langkah 1: Melanjutkan Proyek bank-mini

Code:

```
1 package Jobsheet3.id.ac.polinema;
2
3 public class Account {
4     public String ownerName;
5     public double balance;
6
7     public Account(String ownerName, double balance) {
8         this.ownerName = ownerName;
9         this.balance = balance;
10    }
11
12    public void deposit(double amount) {
13        balance = balance + amount;
14    }
15
16    public void withdraw(double amount) {
17        balance = balance - amount;
18        if (isOverDrawn()) {
19            balance = balance + amount;
20            System.out.println("Withdrawal rejected. Insufficient balance.");
21        }
22    }
23
24    public void printInfo() {
25        System.out.println(ownerName + "- Balance: " + balance);
26    }
27
28    public String formatBalance() {
29        return String.format("%.2f", balance);
30    }
31
32    public boolean isOverDrawn() {
33        return balance < 0;
34    }
35
36    public void transferTo(Account target, double amount) {
37        if (this.balance >= amount) {
38            this.withdraw(amount);
39            target.deposit(amount);
40        } else {
41            this.withdraw(amount);
42        }
43    }
44 }
45
```

```
1 package Jobsheet3.id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         // =====
6         Account[] accounts = new Account[3];
7
8         accounts[0] = new Account("nadia", 500000);
9         accounts[0].withdraw(150000);
10
11         accounts[1] = new Account("budi", 1000000);
12
13         accounts[2] = new Account("sari", 750000);
14         accounts[2].withdraw(250000);
15
16         for (Account acc : accounts) {
17             acc.printInfo();
18         }
19     }
20 }
```

Hasil:



```
1 package Jobsheet3.id.ac.polinema;
2
3 import java.util.Scanner;
4
5 public class Main {
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8         Account[] accounts = new Account[3];
9
10        accounts[0] = new Account("Nadia", "Nadia", 500000);
11        accounts[1] = new Account("Budi", "Budi", 100000);
12        accounts[2] = new Account("Sari", "Sari", 200000);
13
14        for (Account acc : accounts) {
15            acc.printInfo();
16        }
17
18        // Simulasi Transaksi
19        // Account yang akan diambil saldo
20        Account acc = accounts[0];
21        // Simulasi Transaksi
22        acc.withdraw(100000);
23        // Simulasi Transaksi
24        acc.printInfo();
25    }
26}
```

Penjelasan: Pada langkah ini, kita melanjutkan proyek bank-mini dengan membuat *Array of Objects* di dalam class Main. Kita membuat satu wadah (array) bernama accounts yang bisa menyimpan tiga data akun bank sekaligus (Nadia, Budi, dan Sari). Setiap akun diisi saldonya lalu beberapa di antaranya melakukan penarikan uang menggunakan fungsi withdraw(). Terakhir, kita menggunakan perulangan (*for-loop*) untuk mencetak informasi semua akun tersebut secara otomatis ke layar.

Langkah 2: Menerapkan Encapsulation ke Account

Code:



```
1 package Jobsheet3.id.ac.polinema;
2
3 public class Account {
4     private String accountNumber;
5     private String ownerName;
6     private double balance;
7
8     public Account(String accountNumber, String ownerName, double balance) {
9         this.accountNumber = accountNumber;
10        this.ownerName = ownerName;
11        this.balance = balance;
12    }
13
14    public String getAccountNumber(){
15        return accountNumber;
16    }
17
18    public String getOwnerName(){
19        return ownerName;
20    }
21
22    public double getBalance(){
23        return balance;
24    }
25
26    public boolean deposit(double amount) {
27        if(amount <= 0){
28            return false;
29        }
30        balance += amount;
31        return true;
32    }
33
34    public boolean withdraw(double amount) {
35        if (amount <= 0 || amount > balance) {
36            return false;
37        }
38        balance -= amount;
39        return true;
40    }
41
42    public void printInfo() {
43        System.out.println(accountNumber + " - " + ownerName + " - Balance: " + balance);
44    }
45
46 }
```

```
1 package Jobsheet3.id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account acc= new Account("A001","Nadia", 500000);
6         acc.withdraw(150000);
7         acc.printInfo();
8     }
9 }
```

Hasil:

```
sari- Balance: 500000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-> c; cd 'c:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-' & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:54252' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\36f0f1e97c78cbdefd3fa84552efae14\redhat.java\jdt_ws\Praktikum-Pemrograman-Berbasi-Object-PBO-_17729a2a\bin' 'Jobsheet3.id.ac.polinema.Main'
A001 - Nadia - Balance: 350000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-> |
```

Penjelasan: Langkah ini berfokus pada pengamanan data (Enkapsulasi). Semua data penting di class Account (seperti nomor rekening, nama, dan saldo) dikunci dengan kata kunci private. Artinya, file lain (seperti Main.java) tidak bisa lagi mengubah data ini sembarangan. Sebagai gantinya, kita membuat "jalur resmi" berupa fungsi *Getter* (seperti `getAccountNumber`) untuk sekadar melihat data, dan memodifikasi fungsi transaksi (deposit dan withdraw) agar mengembalikan nilai boolean (`true/false`) sebagai laporan apakah transaksi tersebut sukses atau gagal karena melanggar aturan.

D. Tugas dan Deliverable

- Screenshot output program setelah Langkah 2.

```
sari- Balance: 500000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Ob
R 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO
spend=y,address=localhost:54252' '-XX:+ShowCodeDetailsInExceptionMessages' '-
\redhat.java\jdt_ws\Praktikum-Pemrograman-Berbasi-Object-PBO-_17729a2a\bin' '
A001 - Nadia - Balance: 350000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Ob
```

- Tugas mandiri:
1. Account yang kamu buat belum membatasi jumlah penarikan per hari. Tambahkan atribut privat `dailyWithdrawalLimit` (diisi lewat constructor), lalu ubah `withdraw()` agar juga menolak penarikan yang melebihi batas ini, selain aturan saldo yang sudah ada:

```

1  package Jobsheet3.id.ac.polinema;
2
3  public class Account {
4      private String accountNumber;
5      private String ownerName;
6      private double balance;
7      private double dailyWithdrawalLimit;
8
9      public Account(String accountNumber, String ownerName, double balance, double dailyWithdrawalLimit) {
10         this.accountNumber = accountNumber;
11         this.ownerName = ownerName;
12         this.balance = balance;
13         this.dailyWithdrawalLimit = dailyWithdrawalLimit;
14     }
15
16     public String getAccountNumber(){
17         return accountNumber;
18     }
19
20     public String getOwnerName(){
21         return ownerName;
22     }
23
24     public double getBalance(){
25         return balance;
26     }
27
28     public boolean deposit(double amount) {
29         if(amount<=0){
30             return false;
31         }
32         balance += amount;
33         return true;
34     }
35
36     public boolean withdraw(double amount) {
37         if (amount <= 0 || amount > balance || amount > dailyWithdrawalLimit) {
38             return false;
39         }
40         balance -= amount;
41         return true;
42     }
43
44     public void printInfo() {
45         System.out.println(accountNumber + " - " + ownerName + " - Balance: " + balance);
46     }
47
48 }

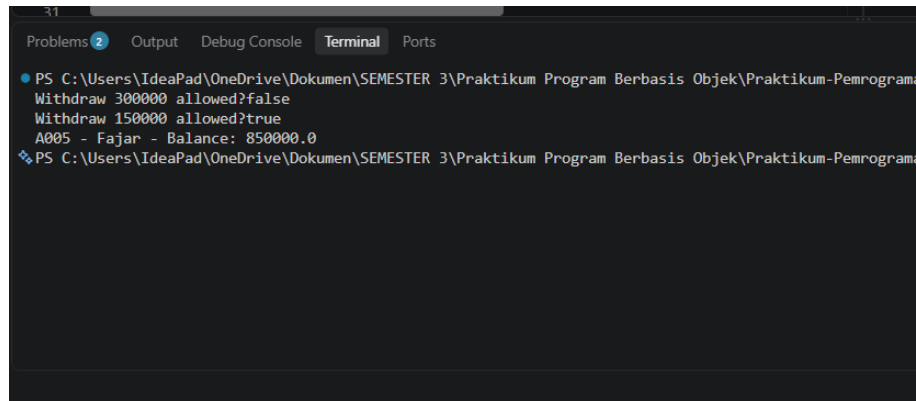
```

```

1  package Jobsheet3.id.ac.polinema;
2
3  public class Main {
4      public static void main(String[] args) {
5          Account limited = new Account("A005", "Fajar", 1000000, 200000);
6          System.out.println("Withdraw 300000 allowed?" + limited.withdraw(300000));
7          System.out.println("Withdraw 150000 allowed?" + limited.withdraw(150000));
8          limited.printInfo();
9      }
10 }

```

Buktikan dengan membuat satu Account bersaldo 1000000 dan batas harian 200000 di Main, lalu coba tarik 300000 (harus ditolak) dan 150000 (harus berhasil):

A screenshot of a Visual Studio Code terminal window. The terminal shows the following output:

```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman> Withdraw 300000 allowed?false
Withdraw 150000 allowed?true
A005 - Fajar - Balance: 850000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman>
```

The terminal window has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active.

Penjelasan: Pada tugas mandiri ini, kita menambahkan aturan baru yaitu batas maksimal penarikan uang dalam sehari (`dailyWithdrawalLimit`). Data ini juga dikunci dengan `private` dan diisi nilainya saat akun pertama kali dibuat melalui Konstruktors. Kemudian, logika di dalam fungsi `withdraw()` diperbarui: sistem sekarang akan menolak penarikan (mengembalikan nilai `false`) jika jumlah uang yang ditarik melebihi batas harian tersebut, meskipun saldonya sebenarnya masih cukup.

2. Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan):
 - a. mengapa mengembalikan nilai boolean dari `deposit()/withdraw()` lebih aman dibandingkan tidak memberi tahu pemanggil sama sekali ketika nilainya ditolak?

Jawab: Mengembalikan nilai boolean bertindak sebagai laporan status transaksi langsung ke sistem pemanggil. Jika tidak ada laporan (*void*), sistem luar mungkin mengira transaksi berhasil padahal sebenarnya ditolak, yang bisa menyebabkan kerusakan logika dan kesalahan perhitungan di bagian program lainnya.

- b. sebutkan satu atribut pada `Account` yang menurutmu sebaiknya dibuat `read-only` (getter tanpa setter), dan jelaskan alasannya.

Jawab: Atribut `accountNumber` (nomor rekening) adalah contoh terbaik untuk dibuat *read-only*. Alasannya, nomor rekening adalah identitas unik yang hanya boleh ditetapkan satu kali saat pembuatan akun di awal dan tidak boleh bisa diubah lagi di kemudian hari demi keamanan data nasabah.

Hasil dan Pembahasan

Praktikum ini berhasil membuktikan kemampuan konsep Enkapsulasi murni pada sistem bank-mini. Dengan menyembunyikan data menggunakan *access modifier* *private*, atribut penting seperti saldo (*balance*) dan batas harian (*dailyWithdrawalLimit*) terbukti sangat aman dari manipulasi langsung oleh *file* lain. Semua perubahan data dipaksa melewati metode resmi (*withdraw* dan *deposit*) yang sudah dilengkapi aturan ketat. Hasil pengujian di terminal menunjukkan bahwa logika pembatasan berfungsi sempurna: penarikan 300.000 ditolak (*false*) karena melebihi batas harian 200.000, sedangkan penarikan 150.000 sukses dieksekusi (*true*) dan memotong saldo dengan tepat.

Kesimpulan

Kesimpulan dari Jobsheet 3 ini adalah bahwa Enkapsulasi (*private*) merupakan benteng pertahanan paling utama dalam Pemrograman Berorientasi Objek. Menggembok data dan hanya menyediakan fungsi khusus (seperti *Getter*) tidak cuma mencegah *human error* atau peretasan data dari luar. Hal ini juga menjamin bahwa setiap perubahan data yang terjadi di dalam program selalu melewati proses pengecekan syarat yang ketat, sehingga aplikasi menjadi jauh lebih stabil, logis, dan tidak mudah rusak saat dikembangkan bersama tim.

Lampiran

[Link Github Ahmad Rafid Riqkullah](https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet3/id/ac/polinema) atau <https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet3/id/ac/polinema>